

Билет 2. Синхронный многопоточный TCP-сервер. TCP-клиент

0. Формулировка билета

Реализация синхронного многопоточного TCP-сервера. Реализация TCP-клиента.

Главная идея билета:

1. Понять, как из однопоточного TCP-сервера сделать сервер, который может обслуживать несколько клиентов одновременно.
2. Понять, как клиент создаёт TCP-сокеты, подключается к серверу через `connect` и обменивается данными через `read` / `write`.
3. Уметь объяснить ограничения подхода "один поток на клиента".

Ключевая схема:

```
server:
socket → bind → listen → accept
                        ↓
                    std::thread
                        ↓
                handle_client(client_fd)
```

Клиентская схема:

```
client:
socket → connect → write/read → close
```

1. Откуда возникает многопоточность

В билете 1 был однопоточный сервер:

```
accept client #1
handle client #1
close client #1

accept client #2
handle client #2
close client #2
```

Проблема:

пока сервер обслуживает client #1,
он не может обслуживать client #2

Если первый клиент подключился и молчит, сервер может зависнуть на:

```
read(client_fd, buffer, sizeof(buffer));
```

и не вернуться к `accept`.

Чтобы этого избежать, можно разделить обязанности:

главный поток:
принимает новых клиентов через `accept`

рабочие потоки:
обслуживают конкретных клиентов

Схема:



Теперь главный поток может быстро вернуться к `accept`:

```
main thread:  
accept client #1 → создать thread #1  
accept client #2 → создать thread #2  
accept client #3 → создать thread #3  
...
```

А обработка клиентов идёт параллельно:

```
thread #1: handle client #1
thread #2: handle client #2
thread #3: handle client #3
```

2. Что значит “синхронный многопоточный сервер”

Слово **синхронный** здесь означает, что операции ввода-вывода остаются блокирующими.

Например:

```
read(client_fd, buffer, sizeof(buffer));
```

если данных нет, блокирует поток.

```
accept(server_fd, ...);
```

если клиентов нет, блокирует поток.

Но за счёт многопоточности блокируется не весь сервер, а только конкретный поток.

Сравнение:

Модель	Что происходит при медленном клиенте
Однопоточный сервер	весь сервер зависает на этом клиенте
Многопоточный сервер	зависает только поток этого клиента
Non-blocking/epoll	один поток может обслуживать много fd без блокировки на одном клиенте

Важно:

```
многопоточный ≠ неблокирующий
```

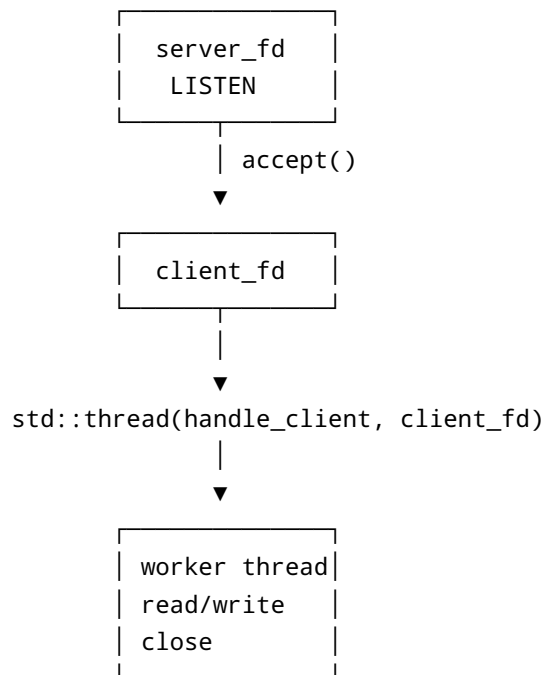
В этом билете сервер многопоточный, но I/O всё ещё blocking.

3. Базовая архитектура “один поток на клиента”

Алгоритм:

1. Создать `server_fd`.
2. `bind`.
3. `listen`.
4. В цикле:
 - 4.1. `accept` возвращает `client_fd`.
 - 4.2. Создаём новый `std::thread`.
 - 4.3. Передаём `client_fd` в `handle_client`.
 - 4.4. Главный поток возвращается к `accept`.

Схема:



4. Минимальная функция `handle_client`

Функция обработки клиента получает `client_fd`.

Простейший вариант:

```
void handle_client(int client_fd) {
    char buffer[1024];

    ssize_t n = read(client_fd, buffer, sizeof(buffer));

    if (n > 0) {
        write(client_fd, buffer, n);
    }
}
```

```
    close(client_fd);  
}
```

Это уже echo-сервер: он отправляет клиенту обратно то, что получил.

Но вариант выше слишком простой:

- `read` вызывается один раз;
- `write` может записать не все байты;
- нет обработки ошибок;
- клиент закрывается сразу после одного сообщения.

Более нормальная версия:

```
void handle_client(int client_fd) {  
    char buffer[4096];  
  
    while (true) {  
        ssize_t n = read(client_fd, buffer, sizeof(buffer));  
  
        if (n < 0) {  
            if (errno == EINTR) {  
                continue;  
            }  
  
            std::cerr << "read failed: " << strerror(errno) << "\n";  
            break;  
        }  
  
        if (n == 0) {  
            std::cout << "client disconnected\n";  
            break;  
        }  
  
        ssize_t written = 0;  
  
        while (written < n) {  
            ssize_t res = write(client_fd, buffer + written, n - written);  
  
            if (res < 0) {  
                if (errno == EINTR) {  
                    continue;  
                }  
  
                std::cerr << "write failed: " << strerror(errno) << "\n";  
                close(client_fd);  
                return;  
            }  
  
            written += res;  
        }  
    }  
}
```

```

        if (res == 0) {
            close(client_fd);
            return;
        }

        written += res;
    }
}

close(client_fd);
}

```

Что важно:

```

n > 0   — прочитали n байт
n == 0  — клиент закрыл соединение
n < 0   — ошибка

```

5. Вспомогательная функция `write_all`

Поскольку `write` может записать меньше байт, чем мы попросили, удобно вынести цикл в отдельную функцию.

```

bool write_all(int fd, const char* data, size_t size) {
    size_t written = 0;

    while (written < size) {
        ssize_t res = write(fd, data + written, size - written);

        if (res < 0) {
            if (errno == EINTR) {
                continue;
            }

            return false;
        }

        if (res == 0) {
            return false;
        }

        written += static_cast<size_t>(res);
    }

    return true;
}

```

Тогда `handle_client` становится чище:

```
void handle_client(int client_fd) {
    char buffer[4096];

    while (true) {
        ssize_t n = read(client_fd, buffer, sizeof(buffer));

        if (n < 0) {
            if (errno == EINTR) {
                continue;
            }

            break;
        }

        if (n == 0) {
            break;
        }

        if (!write_all(client_fd, buffer, static_cast<size_t>(n))) {
            break;
        }
    }

    close(client_fd);
}
```

6. Базовый многопоточный TCP echo-сервер

```
#include <arpa/inet.h>
#include <cerrno>
#include <cstring>
#include <iostream>
#include <thread>
#include <unistd.h>
#include <sys/socket.h>

bool write_all(int fd, const char* data, size_t size) {
    size_t written = 0;

    while (written < size) {
        ssize_t res = write(fd, data + written, size - written);

        if (res < 0) {
            if (errno == EINTR) {
                continue;
            }
        }
    }
}
```

```

        }

        return false;
    }

    if (res == 0) {
        return false;
    }

    written += static_cast<size_t>(res);
}

return true;
}

void handle_client(int client_fd, int client_id) {
    char buffer[4096];

    std::cout << "client #" << client_id << " handler started\n";

    while (true) {
        ssize_t n = read(client_fd, buffer, sizeof(buffer));

        if (n < 0) {
            if (errno == EINTR) {
                continue;
            }

            std::cerr << "read failed: " << strerror(errno) << "\n";
            break;
        }

        if (n == 0) {
            std::cout << "client #" << client_id << " disconnected\n";
            break;
        }

        if (!write_all(client_fd, buffer, static_cast<size_t>(n))) {
            std::cerr << "write failed\n";
            break;
        }
    }

    close(client_fd);
}

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);

    if (server_fd < 0) {
        std::cerr << "socket failed: " << strerror(errno) << "\n";
    }
}

```



```

        return 1;
    }

    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(11000);
    addr.sin_addr.s_addr = INADDR_ANY;

    if (bind(server_fd, reinterpret_cast<sockaddr*>(&addr), sizeof(addr)) <
0) {
        std::cerr << "bind failed: " << strerror(errno) << "\n";
        close(server_fd);
        return 1;
    }

    if (listen(server_fd, 10) < 0) {
        std::cerr << "listen failed: " << strerror(errno) << "\n";
        close(server_fd);
        return 1;
    }

    std::cout << "server is listening on port 11000\n";

    int client_counter = 0;

    while (true) {
        sockaddr_in client_addr{};
        socklen_t client_len = sizeof(client_addr);

        int client_fd = accept(
            server_fd,
            reinterpret_cast<sockaddr*>(&client_addr),
            &client_len
        );

        if (client_fd < 0) {
            if (errno == EINTR) {
                continue;
            }

            std::cerr << "accept failed: " << strerror(errno) << "\n";
            continue;
        }

        char ip[INET_ADDRSTRLEN];

        inet_ntop(
            AF_INET,

```

```

        &client_addr.sin_addr,
        ip,
        sizeof(ip)
    );

    int client_id = client_counter++;

    std::cout << "new client #" << client_id
               << " from " << ip
               << ":" << ntohs(client_addr.sin_port)
               << "\n";

    std::thread t(handle_client, client_fd, client_id);
    t.detach();
}

close(server_fd);
}

```

7. Что происходит в этом сервере

Главный поток делает только:

```

accept
create thread
accept
create thread
accept
create thread
...

```

Каждый клиент обрабатывается в отдельном потоке:

```

thread #1:
    read/write client #1

thread #2:
    read/write client #2

thread #3:
    read/write client #3

```

Главный поток не ждёт, пока клиент закончит общение.

Именно это даёт параллельность:

один клиент молчит → блокируется только его worker thread
другие клиенты продолжают обслуживаться своими потоками

8. `std::thread`

`std::thread` создаёт новый поток выполнения.

Пример:

```
std::thread t(handle_client, client_fd, client_id);
```

Это означает:

запустить функцию `handle_client(client_fd, client_id)` в новом потоке

Аргументы копируются или перемещаются в новый поток.

В нашем случае:

```
client_fd  
client_id
```

копируются как обычные `int`.

Это хорошо: если бы мы передали ссылку на переменную, которую главный поток потом изменит, можно было бы получить ошибку.

9. Важная ошибка: передача ссылки на `client_fd`

Неправильно:

```
while (true) {  
    int client_fd = accept(server_fd, nullptr, nullptr);  
  
    std::thread t([&] {  
        handle_client(client_fd);  
    });  
  
    t.detach();  
}
```

Почему опасно?

Потому что лямбда захватывает `client_fd` по ссылке:

```
[&]
```

А главный поток продолжит цикл и изменит `client_fd`.

В результате несколько потоков могут увидеть не тот fd.

Правильно:

```
int client_fd = accept(server_fd, nullptr, nullptr);

std::thread t([client_fd] {
    handle_client(client_fd);
});

t.detach();
```

Или проще:

```
std::thread t(handle_client, client_fd);
t.detach();
```

Здесь `client_fd` передаётся по значению.

10. `join` и `detach`

У объекта `std::thread` есть два основных способа завершения владения потоком:

```
t.join();
```

или

```
t.detach();
```

`join`

`join` ждёт завершения потока.

```
std::thread t(handle_client, client_fd);  
t.join();
```

Но если сделать `join` сразу после создания потока в сервере, мы потеряем параллельность:

```
while (true) {  
    int client_fd = accept(server_fd, ...);  
  
    std::thread t(handle_client, client_fd);  
    t.join(); // главный поток ждёт клиента  
}
```

Это почти то же самое, что однопоточный сервер:

```
accept client #1  
create thread #1  
wait thread #1  
accept client #2  
create thread #2  
wait thread #2  
...
```

`detach`

`detach` отделяет поток от объекта `std::thread`.

```
std::thread t(handle_client, client_fd);  
t.detach();
```

После `detach` поток работает самостоятельно.

Плюс:

главный поток может сразу вернуться к `accept`

Минусы:

нельзя дождаться завершения потока
сложнее корректно остановить сервер
сложнее обрабатывать ошибки
сложнее управлять ресурсами

Для учебного сервера `detach` допустим.

Для серьёзного сервера чаще используют:

- пул потоков;
- список потоков и `join`;
- `std::jthread`;
- event loop;
- асинхронный I/O.

11. Важное правило `std::thread`: нельзя уничтожать joinable thread

Если объект `std::thread` уничтожается, пока поток ещё joinable, вызывается `std::terminate`.

Например:

```
{
    std::thread t(handle_client, client_fd);
} // плохо: t уничтожается, но ни join, ни detach не вызваны
```

Это приведёт к аварийному завершению программы.

Поэтому после создания потока нужно вызвать:

```
t.join();
```

или:

```
t.detach();
```

В учебном сервере обычно используют:

```
std::thread t(handle_client, client_fd);
t.detach();
```

12. Вариант с хранением потоков и `join`

Можно не делать `detach`, а хранить потоки в `std::vector`.

```
std::vector<std::thread> threads;
```

```
while (true) {  
    int client_fd = accept(server_fd, nullptr, nullptr);  
  
    threads.emplace_back(handle_client, client_fd);  
}
```

Но здесь появляется проблема:

когда делать join?

Если сервер бесконечный, до `join` дело может не дойти.

Можно при завершении сервера пройтись:

```
for (auto& t : threads) {  
    if (t.joinable()) {  
        t.join();  
    }  
}
```

Но для этого нужен механизм остановки сервера.

В учебном примере `detach` проще.

13. Дополнение: `std::jthread`

В C++20 есть `std::jthread`.

Он отличается от `std::thread` тем, что в деструкторе автоматически делает `join`.

```
std::jthread t(handle_client, client_fd);
```

Плюс:

меньше риска забыть join/detach

Минус:

если `jthread` живёт только до конца итерации цикла,
его деструктор сразу сделает `join`,
и сервер снова станет последовательным

Поэтому `std::jthread` полезен, но его всё равно надо правильно хранить.

14. TCP-клиент

Клиентская последовательность проще серверной:

```
socket()
↓
connect()
↓
write()
↓
read()
↓
close()
```

В отличие от сервера, клиент обычно не вызывает:

```
bind()
listen()
accept()
```

Почему?

Потому что клиент сам инициирует соединение к известному серверу.

Обычно ядро автоматически выбирает клиенту локальный эфемерный порт.

15. Минимальный TCP-клиент

```
#include <arpa/inet.h>
#include <cerrno>
#include <cstring>
#include <iostream>
#include <sys/socket.h>
#include <unistd.h>

int main() {
    int fd = socket(AF_INET, SOCK_STREAM, 0);

    if (fd < 0) {
        std::cerr << "socket failed: " << strerror(errno) << "\n";
        return 1;
    }

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
```



```

addr.sin_port = htons(11000);

if (inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr) <= 0) {
    std::cerr << "bad address\n";
    close(fd);
    return 1;
}

if (connect(fd, reinterpret_cast<sockaddr*>(&addr), sizeof(addr)) < 0) {
    std::cerr << "connect failed: " << strerror(errno) << "\n";
    close(fd);
    return 1;
}

const char* message = "hello from client\n";

write(fd, message, strlen(message));

char buffer[1024];

ssize_t n = read(fd, buffer, sizeof(buffer) - 1);

if (n > 0) {
    buffer[n] = '\0';
    std::cout << "server replied: " << buffer;
}

close(fd);
}

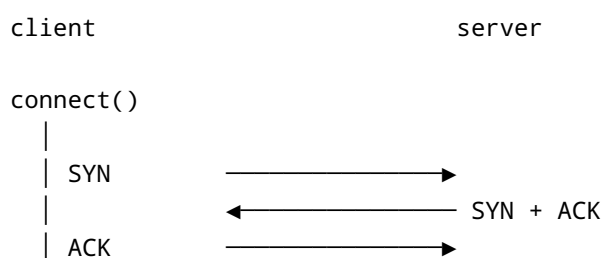
```

16. connect()

`connect` инициирует TCP-соединение с сервером.

```
connect(fd, reinterpret_cast<sockaddr*>(&addr), sizeof(addr));
```

Что происходит логически:



▼
connect returns

После успешного `connect` можно делать:

```
write(fd, ...);  
read(fd, ...);
```

Если сервер не слушает порт, `connect` может завершиться ошибкой.

Например:

ECONNREFUSED

Если сеть недоступна:

ENETUNREACH

Если соединение долго не устанавливается:

ETIMEDOUT

17. `inet_addr` и `inet_pton`

В учебном коде часто встречается:

```
addr.sin_addr.s_addr = inet_addr("127.0.0.1");
```

Это работает для IPv4, но более современный вариант:

```
inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);
```

Почему `inet_pton` лучше:

- поддерживает IPv4 и IPv6
- лучше сообщает об ошибках
- не считается устаревшим вариантом

Расшифровка:

```
pton = presentation to network
```

То есть из строкового представления адреса в сетевой бинарный формат.

Обратная функция:

```
inet_ntop
```

Она переводит бинарный адрес в строку.

18. Клиент с вводом из stdin

Учебный клиент может читать строки из стандартного ввода и отправлять их серверу.

Плохой вариант:

```
char buf[1024];  
scanf("%s", buf);
```

Проблемы:

```
scanf("%s")
```

 без ограничения длины может переполнить буфер
читает только до пробела

Лучше использовать `std::string`:

```
#include <iostream>  
#include <string>  
  
std::string line;  
  
while (std::getline(std::cin, line)) {  
    line.push_back('\n');  
    write_all(fd, line.data(), line.size());  
  
    char buffer[1024];  
    ssize_t n = read(fd, buffer, sizeof(buffer) - 1);  
  
    if (n <= 0) {  
        break;  
    }  
  
    buffer[n] = '\0';
```

```
std::cout << "server: " << buffer;
}
```

Ещё лучше — явно договориться о протоколе сообщений.

19. Клиент с `write_all`

```
#include <arpa/inet.h>
#include <cerrno>
#include <cstring>
#include <iostream>
#include <string>
#include <sys/socket.h>
#include <unistd.h>

bool write_all(int fd, const char* data, size_t size) {
    size_t written = 0;

    while (written < size) {
        ssize_t res = write(fd, data + written, size - written);

        if (res < 0) {
            if (errno == EINTR) {
                continue;
            }

            return false;
        }

        if (res == 0) {
            return false;
        }

        written += static_cast<size_t>(res);
    }

    return true;
}

int main() {
    int fd = socket(AF_INET, SOCK_STREAM, 0);

    if (fd < 0) {
        std::cerr << "socket failed: " << strerror(errno) << "\n";
        return 1;
    }

    sockaddr_in addr{};
```

```

addr.sin_family = AF_INET;
addr.sin_port = htons(11000);

if (inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr) <= 0) {
    std::cerr << "bad address\n";
    close(fd);
    return 1;
}

if (connect(fd, reinterpret_cast<sockaddr*>(&addr), sizeof(addr)) < 0) {
    std::cerr << "connect failed: " << strerror(errno) << "\n";
    close(fd);
    return 1;
}

std::string line;

while (std::getline(std::cin, line)) {
    line.push_back('\n');

    if (!write_all(fd, line.data(), line.size())) {
        std::cerr << "write failed\n";
        break;
    }

    char buffer[1024];

    ssize_t n = read(fd, buffer, sizeof(buffer) - 1);

    if (n < 0) {
        if (errno == EINTR) {
            continue;
        }

        std::cerr << "read failed: " << strerror(errno) << "\n";
        break;
    }

    if (n == 0) {
        std::cout << "server closed connection\n";
        break;
    }

    buffer[n] = '\0';
    std::cout << "server: " << buffer;
}

close(fd);
}

```

20. Важный момент: сервер может закрывать соединение после одного ответа

Если клиент написан так:

```
while (true) {  
    write(fd, ...);  
    read(fd, ...);  
}
```

а сервер после одного ответа делает:

```
close(client_fd);
```

то клиент сможет отправить только одно сообщение.

При следующей попытке чтения/записи он получит:

```
read == 0
```

или ошибку записи.

Поэтому поведение клиента и сервера должно совпадать.

Возможные варианты протокола:

Вариант 1. Одно сообщение на соединение

```
client connects  
client sends one request  
server sends one response  
server closes connection
```

Вариант 2. Много сообщений на одном соединении

```
client connects  
while connected:  
    client sends request  
    server sends response
```

Для второго варианта `handle_client` должен содержать цикл `read/write`.

21. Проблема общего состояния

Многопоточность становится особенно важной, когда потоки имеют общие данные.

Например, сервер проверки простоты чисел может иметь общий кэш:

```
std::unordered_map<long long, bool> cache;
```

Если несколько потоков одновременно обращаются к этому кэшу без синхронизации, возникает data race.

Плохо:

```
std::unordered_map<long long, bool> cache;

bool get_from_cache(long long x) {
    auto it = cache.find(x);

    if (it != cache.end()) {
        return it->second;
    }

    bool result = is_prime(x);
    cache[x] = result;
    return result;
}
```

Если эту функцию одновременно вызывают разные потоки, то `unordered_map` может быть повреждён.

Правильно использовать мьютекс:

```
std::unordered_map<long long, bool> cache;
std::mutex cache_mutex;

bool get_from_cache(long long x) {
    {
        std::lock_guard<std::mutex> lock(cache_mutex);

        auto it = cache.find(x);

        if (it != cache.end()) {
            return it->second;
        }
    }

    bool result = is_prime(x);
```

```

    {
        std::lock_guard<std::mutex> lock(cache_mutex);
        cache[x] = result;
    }

    return result;
}

```

Но здесь есть тонкость: два потока могут одновременно не найти значение, оба посчитать `is_prime`, а потом оба записать один и тот же результат.

Это не data race, если запись защищена мьютексом, но может быть лишняя работа.

Для билета 2 достаточно понимать:

если у потоков есть общие изменяемые данные,
нужна синхронизация

Подробно это будет в билете 3 про mutex.

22. Конкуренция за `std::cout`

В конспекте есть замечание: несколько потоков могут одновременно писать в `std::cout`.

Например:

```
std::cout << "client " << id << " disconnected\n";
```

Если два потока пишут одновременно, вывод может перемешаться:

```
client client 1 disconnected
2 disconnected
```

Это не обязательно приводит к неопределённому поведению для стандартных потоков ввода-вывода, но лог становится нечитаемым.

Можно защитить вывод мьютексом:

```
std::mutex cout_mutex;

void log(const std::string& message) {
    std::lock_guard<std::mutex> lock(cout_mutex);

```



```
std::cout << message << std::endl;  
}
```

Но в промышленном коде лучше использовать нормальную систему логирования.

23. Проблема C10k

Подход “один поток на клиента” простой, но плохо масштабируется.

C10k — это задача обслужить 10 000 одновременных клиентов.

Почему поток на каждого клиента плохо работает при большом числе соединений?

1. Память под стек

У каждого потока есть стек.

Если стек по умолчанию 8 MiB, то:

```
10 000 потоков × 8 MiB = 80 GiB
```

Это слишком много.

2. Накладные расходы планировщика

ОС должна планировать тысячи потоков.

Даже если многие из них спят в `read`, управление таким количеством потоков стоит ресурсов.

3. Context switch

Переключение между потоками требует времени.

Нужно сохранить состояние одного потока и восстановить состояние другого.

4. Ресурсы ядра

Каждый поток требует структур ядра.

Каждое соединение требует файловый дескриптор.

У процесса есть лимиты:

```
ulimit -n
```

и системные лимиты на количество потоков.

5. Нет backpressure на уровне приложения

Если новые клиенты приходят быстрее, чем сервер их обслуживает, сервер продолжит создавать новые потоки до истощения ресурсов.

24. Альтернативы “поток на клиента”

Эта тема связывает билет 2 с последующими билетами.

Thread pool

Вместо бесконечного создания потоков создаём фиксированное число worker threads.

```
main thread:
  accept client_fd
  put client_fd into queue

worker threads:
  take client_fd from queue
  handle_client(client_fd)
```

Плюсы:

```
ограничиваем число потоков
лучше контролируем ресурсы
```

Минусы:

```
если handle_client блокируется на долгий read,
воркер занят этим клиентом
```

Thread pool подробно будет в билете 5.

Non-blocking I/O + epoll

Один или несколько потоков могут обслуживать много соединений через события готовности.

```
epoll_wait
↓
fd ready
↓
read/write without blocking
```

Плюсы:

лучше масштабируется на большое число соединений

Минусы:

сложнее код
надо хранить состояния соединений

Это будет в билетах 10–12.

Coroutines / fibers

Можно писать код в синхронном стиле, но исполнять его поверх неблокирующего I/O или пользовательского scheduler.

Это будет в билетах 15–16.

25. SIGPIPE и запись в закрытый сокет

Если клиент закрыл соединение, а сервер пытается сделать:

```
write(client_fd, data, size);
```

процесс может получить SIGPIPE.

По умолчанию SIGPIPE может завершить процесс.

В учебных примерах это часто опускают, но профессионально надо знать.

Варианты решения:

1. Игнорировать SIGPIPE

```
#include <signal.h>

signal(SIGPIPE, SIG_IGN);
```

2. Использовать send c MSG_NOSIGNAL

```
send(client_fd, data, size, MSG_NOSIGNAL);
```

3. Обработать ошибку EPIPE

```
ssize_t res = write(fd, data, size);

if (res < 0 && errno == EPIPE) {
    // другая сторона закрыла соединение
}
```

26. RAII для файлового дескриптора

В C++ полезно заворачивать fd в RAII-класс, чтобы не забыть `close`.

Пример:

```
class unique_fd {
public:
    unique_fd() = default;

    explicit unique_fd(int fd) : fd_(fd) {}

    ~unique_fd() {
        if (fd_ >= 0) {
            close(fd_);
        }
    }

    unique_fd(const unique_fd&) = delete;
    unique_fd& operator=(const unique_fd&) = delete;

    unique_fd(unique_fd&& other) noexcept : fd_(other.fd_) {
        other.fd_ = -1;
    }

    unique_fd& operator=(unique_fd&& other) noexcept {
        if (this != &other) {
            if (fd_ >= 0) {
                close(fd_);
            }

            fd_ = other.fd_;
            other.fd_ = -1;
        }

        return *this;
    }

    int get() const {
```

```

        return fd_;
    }

    int release() {
        int result = fd_;
        fd_ = -1;
        return result;
    }

private:
    int fd_ = -1;
};

```

Тогда можно писать:

```
unique_fd client(accept(server_fd, nullptr, nullptr));
```

Но при передаче fd в поток нужно внимательно решить, кто владеет fd.

Например:

```

unique_fd client(client_fd);

std::thread t([fd = client.release()] {
    handle_client(fd);
});

t.detach();

```

Идея:

```
после release ответственность за close переходит в handle_client
```

Для экзамена это профессиональное дополнение, не обязательная часть.

27. Ошибки lifetime при передаче fd в поток

Файловый дескриптор — это просто число.

Если главный поток случайно закроет `client_fd`, пока worker thread ещё им пользуется, worker получит невалидный fd.

Плохо:

```
int client_fd = accept(server_fd, nullptr, nullptr);

std::thread t(handle_client, client_fd);
t.detach();

close(client_fd); // плохо: worker ещё использует fd
```

Правильно:

```
int client_fd = accept(server_fd, nullptr, nullptr);

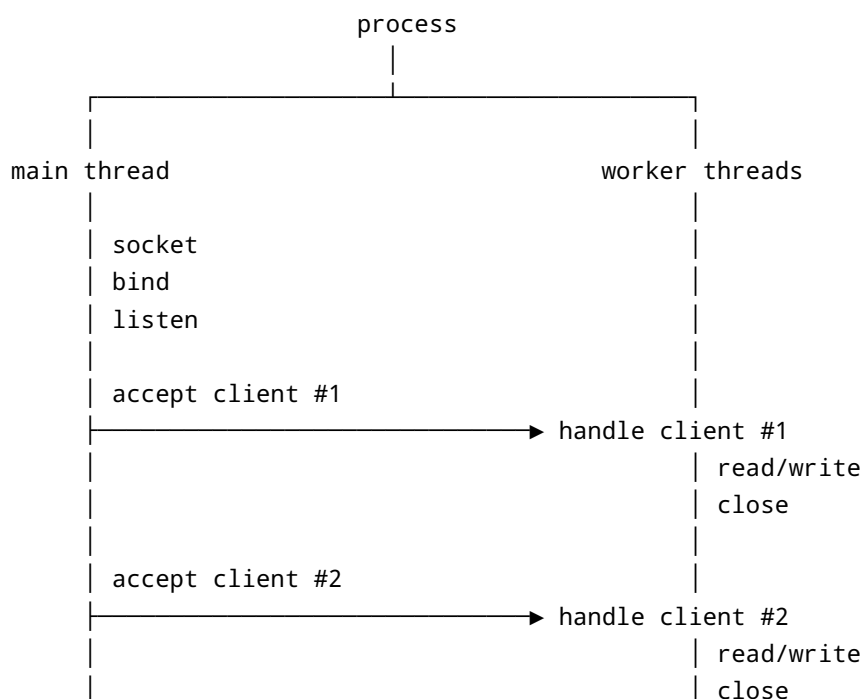
std::thread t(handle_client, client_fd);
t.detach();

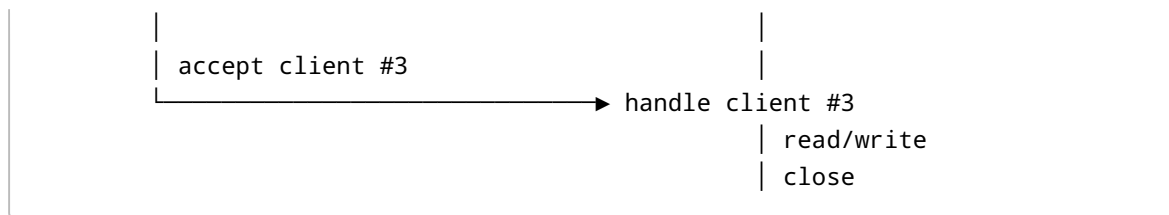
// main thread больше не трогает client_fd
// close делает handle_client
```

Владение должно быть понятным:

ассепт возвращает fd главному потоку
главный поток передаёт fd worker-потоку
worker-поток закрывает fd

28. Полная схема многопоточного сервера





29. Сравнение однопоточного и многопоточного сервера

Критерий	Однопоточный сервер	Поток на клиента
Простота	очень простой	чуть сложнее
Одновременные клиенты	нет	да
Blocking I/O	блокирует весь сервер	блокирует только поток клиента
Расход памяти	маленький	большой при большом числе клиентов
Context switch	мало	может быть много
Общие данные	почти нет проблемы	нужна синхронизация
Масштабируемость	плохая	лучше, но C10k проблема
Подходит для обучения	да	да
Подходит для highload	нет	обычно нет

30. Как тестировать

Запустить сервер

```
g++ -std=c++20 -pthread server.cpp -o server
./server
```

Ключевой флаг:

```
-pthread
```

Он нужен для корректной сборки программы с потоками.

Подключиться через `nc`

В одном терминале:

```
nc localhost 11000
```

Во втором терминале:

```
nc localhost 11000
```

Теперь можно писать в оба соединения.

Если сервер многопоточный, оба клиента должны обслуживаться независимо.

Запустить свой клиент

```
g++ -std=c++20 client.cpp -o client  
./client
```

Посмотреть потоки процесса

Можно найти PID сервера:

```
pgrep server
```

Посмотреть потоки:

```
ps -T -p <pid>
```

Или:

```
ls /proc/<pid>/task
```

31. Типичные ошибки в ответе

Ошибка 1. Сказать, что многопоточный сервер обязательно неблокирующий

Нет.

В этом билете сервер многопоточный, но операции `read`, `write`, `accept` остаются блокирующими.

Ошибка 2. Сделать `join` сразу после создания потока

Плохо:


```
std::thread t(handle_client, client_fd);  
t.join();
```

Если `join` стоит прямо внутри accept loop, главный поток будет ждать завершения клиента.

Параллельность пропадёт.

Ошибка 3. Не вызывать ни `join`, ни `detach`

Плохо:

```
std::thread t(handle_client, client_fd);
```

При уничтожении joinable `std::thread` программа вызовет `std::terminate`.

Ошибка 4. Передать `client_fd` по ссылке

Плохо:

```
std::thread t([&] {  
    handle_client(client_fd);  
});
```

Лучше:

```
std::thread t(handle_client, client_fd);
```

или:

```
std::thread t([client_fd] {  
    handle_client(client_fd);  
});
```

Ошибка 5. Закрывать `client_fd` в главном потоке после передачи в worker

Плохо:

```
std::thread t(handle_client, client_fd);  
t.detach();  
close(client_fd);
```

Если worker должен использовать fd, закрывать его должен worker.

Ошибка 6. Использовать общий unordered_map без мьютекса

В многопоточном сервере общие изменяемые данные требуют синхронизации.

Ошибка 7. Не учитывать partial write

Плохо:

```
write(fd, buffer, n);
```

и считать, что точно записались все `n` байт.

Лучше:

```
write_all(fd, buffer, n);
```

Ошибка 8. Использовать scanf("%s", buf) без ограничения

Это может привести к переполнению буфера.

Лучше:

```
std::getline(std::cin, line);
```

или хотя бы:

```
scanf("%1023s", buf);
```

32. Что сказать на экзамене: короткая версия

Можно отвечать так:

В однопоточном сервере главный недостаток в том, что после `accept` сервер обрабатывает клиента в том же потоке. Если клиент долго не отправляет данные, сервер блокируется на `read` и не может обслуживать остальных.

Простейшее многопоточное решение — после каждого `accept` создавать новый `std::thread`, который получает `client_fd` и вызывает `handle_client(client_fd)`. Главный поток после создания потока возвращается к `accept`, а worker-поток занимается чтением, записью и закрытием конкретного клиентского сокета.

Важно, что `server_fd` остаётся в главном потоке и продолжает слушать порт, а `client_fd` передаётся в поток обработки. Передавать `client_fd` нужно по значению, чтобы не получить ошибку `lifetime`. Закрывать `client_fd` должен тот поток, который его обслуживает.

Для учебного примера можно сделать `std::thread t(handle_client, client_fd); t.detach();`. Если не вызвать ни `join`, ни `detach`, при уничтожении объекта `std::thread` программа завершится через `std::terminate`. Если вызвать `join` сразу внутри цикла `accept`, параллельность исчезнет.

TCP-клиент создаёт сокет через `socket(AF_INET, SOCK_STREAM, 0)`, заполняет адрес сервера, вызывает `connect`, после чего может делать `write/read`. Клиент обычно не вызывает `bind`, потому что ОС сама назначает ему эфемерный порт.

Подход “один поток на клиента” прост, но плохо масштабируется: у каждого потока есть стек, много потоков создают нагрузку на планировщик, появляются context switches и лимиты ресурсов. Поэтому для большого числа соединений переходят к thread pool, неблокирующему I/O, `epoll` или `io_uring`.

33. Мини-код для ответа на доске

Сервер

```
void handle_client(int client_fd) {
    char buffer[1024];

    while (true) {
        ssize_t n = read(client_fd, buffer, sizeof(buffer));

        if (n <= 0) {
            break;
        }

        write(client_fd, buffer, n);
    }

    close(client_fd);
}
```

```

}

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(11000);
    addr.sin_addr.s_addr = INADDR_ANY;

    bind(server_fd, reinterpret_cast<sockaddr*>(&addr), sizeof(addr));
    listen(server_fd, 10);

    while (true) {
        int client_fd = accept(server_fd, nullptr, nullptr);

        std::thread t(handle_client, client_fd);
        t.detach();
    }
}

```

Что объяснить:

main thread занимается accept
 worker thread занимается read/write
 client_fd передаётся по значению
 detach нужен, чтобы thread object не вызвал terminate
 join внутри цикла убил бы параллельность
 close(client_fd) делает worker

Клиент

```

int main() {
    int fd = socket(AF_INET, SOCK_STREAM, 0);

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(11000);
    inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);

    connect(fd, reinterpret_cast<sockaddr*>(&addr), sizeof(addr));

    const char* msg = "hello\n";
    write(fd, msg, strlen(msg));

    char buffer[1024];
    ssize_t n = read(fd, buffer, sizeof(buffer) - 1);

    if (n > 0) {

```

```

        buffer[n] = '\0';
        std::cout << buffer;
    }

    close(fd);
}

```

Что объяснить:

socket создаёт клиентский сокет
 connect устанавливает TCP-соединение с сервером
 после connect можно делать read/write
 клиент обычно не делает bind/listen/accept

34. Мини-проверка

1. В чём проблема однопоточного TCP-сервера?
2. Почему в многопоточном сервере главный поток после `accept` создаёт новый поток?
3. Что делает `handle_client`?
4. Почему `client_fd` нужно передавать в поток по значению?
5. Что будет, если захватить `client_fd` в лямбде по ссылке?
6. Почему нельзя уничтожить joinable `std::thread`?
7. Чем `join` отличается от `detach`?
8. Почему `join` сразу после создания потока уничтожает смысл многопоточности?
9. Кто должен закрывать `client_fd`: главный поток или worker?
10. Почему `write` желательно оборачивать в `write_all`?
11. Что означает `read == 0` внутри `handle_client`?
12. Почему несколько потоков могут перемешивать вывод в `std::cout`?
13. Когда в многопоточном сервере нужен `std::mutex`?
14. Почему общий `unordered_map` без мьютекса опасен?
15. Что такое C10k?
16. Почему 10 000 потоков — это проблема?
17. Что делает TCP-клиент до `connect`?
18. Почему клиент обычно не вызывает `bind`?
19. Что делает `connect`?
20. Чем `inet_pton` лучше `inet_addr`?
21. Что произойдёт, если сервер закрывает соединение после одного ответа, а клиент ждёт долгий диалог?
22. Почему многопоточный сервер из этого билета всё ещё называется синхронным?
23. Чем подход “thread per client” отличается от thread pool?
24. Как протестировать сервер двумя клиентами через `nc`?
25. Зачем при компиляции может понадобиться флаг `-pthread`?

35. Самый короткий ответ, если времени совсем мало

Многопоточный TCP-сервер отличается от однопоточного тем, что главный поток не обслуживает клиента сам. Он вызывает `accept`, получает `client_fd`, создаёт `std::thread(handle_client, client_fd)` и возвращается к ожиданию новых подключений. Worker-поток выполняет `read/write` с конкретным клиентом и закрывает `client_fd`.

`client_fd` нужно передавать в поток по значению. После создания потока нужно вызвать `join` или `detach`; в учебном сервере обычно используют `detach`, иначе главный поток не сможет сразу вернуться к `accept`. Если сделать `join` сразу, сервер снова станет последовательным.

TCP-клиент делает `socket`, заполняет адрес сервера, вызывает `connect`, а затем использует `read/write`. Клиент обычно не вызывает `bind`, потому что ОС сама назначает эфемерный порт.

Главный минус модели “один поток на клиента” — плохая масштабируемость: много потоков требуют много памяти, создают нагрузку на планировщик и приводят к большому числу context switches. Поэтому дальше переходят к thread pool, non-blocking I/O, `epoll` и `io_uring`.